

## Experiment # 04

### Implementation of Gray Level Transformations

#### Objective

1. To implement contrast stretching.
2. To implement gray level slicing
3. To implement negative, power and log transformation

#### Software

- Python Idle 3.7 or 3.8.5
- Pycharm/ Spyder

#### Theory

**Contrast stretching** is a simple image enhancement technique that attempts to improve the contrast in an image by 'stretching' the range of intensity values it contains to span a desired range of values. Normally (min, max)  $\rightarrow$  (0, 255)

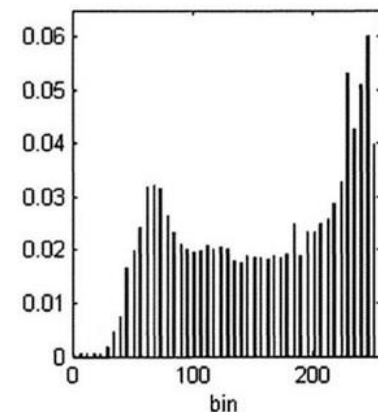
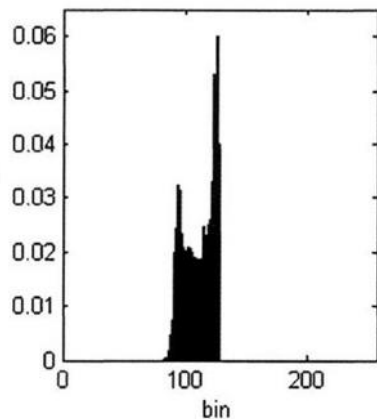
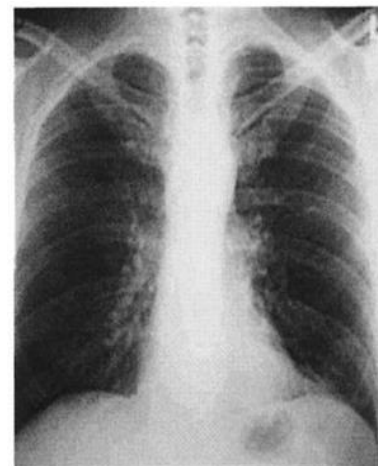


$$p = ((p - \min) / (\max - \min)) * 255$$

Where:

p is pixel value at a certain pixel location

max, min is the max, min pixel value in the entire image



Instead of blindly computing the dynamic range using just the minimum and maximum pixel values, a more robust and adaptive technique is to use the **5th and 95th percentiles** of the input values when deriving the dynamic range of the input image.

The Formula for Contrast stretch or Image Normalization

$$s = \frac{(r - r_{min})(L - 1)}{r_{max} - r_{min}}$$

- r** - Input pixel value
- r<sub>min</sub>** - Minimum pixel value in the input image
- r<sub>max</sub>** - Maximum pixel value in the input image
- L** - Maximum pixel value ( $2^8 = 256$ )

**Python Code:**

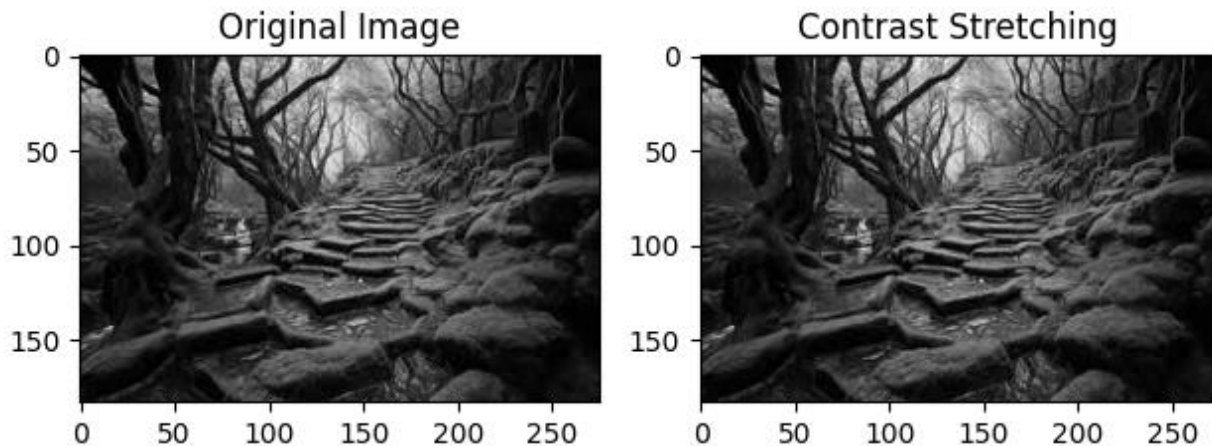
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image in grayscale
image = cv2.imread('download.jpeg', 0)
r_min, r_max = np.min(image), np.max(image)
contrast_stretched = (image - r_min) * (255 / (r_max - r_min))

# Normalize to [0, 255]
contrast_stretched = np.array(contrast_stretched, dtype=np.uint8)

plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')

# Display the result
plt.subplot(1, 2, 2)
plt.imshow(contrast_stretched, cmap='gray')
plt.title('Contrast Stretching')
plt.tight_layout()
plt.show()
```



**Thresholding** is the operation through which an image can be converted into a binary image/black & white i.e. having only two distinct levels. Threshold value can be the mean or median etc. of the image. Each pixel is compared to a threshold value:

- Pixels greater than the threshold are set to one value (e.g., 255).
- Pixels below the threshold are set to another value (e.g., 0).

**Python Code:**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image in grayscale
image = cv2.imread(download.jpeg', 0)

# Apply simple thresholding
# Threshold at 127, values above 127 are set to 255, below are set to 0
_, binary_image = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)

# Apply inverse binary thresholding
_, binary_inv_image = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY_INV)

# Display original and thresholded images
plt.figure(figsize=(10, 5))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(binary_image, cmap='gray')
```

```
plt.title('Binary Thresholding')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(binary_inv_image, cmap='gray')
plt.title('Inverse Binary Thresholding')
plt.axis('off')

plt.tight_layout()
plt.show()
```



**Gray level Slicing** is used to highlight a specific range of gray levels in an image. It can either:

- Highlight a specific intensity range while keeping other intensities intact.
- Highlight a specific intensity range while suppressing all other intensities to black (0).

**Python Code:**

```
def gray_level_slicing(image, min_val, max_val, slice_value):
    # Create an output image filled with zeros (black)
    output_image = np.zeros_like(image)

    # Set the pixel values within the specified range to the slice_value
    output_image[(image >= min_val) & (image <= max_val)] = slice_value

    return output_image

# Apply gray-level slicing to highlight pixels in the range [100, 200]
sliced_image = gray_level_slicing(image, 100, 200, 255)

# Display original and gray-level sliced images
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
plt.imshow(sliced_image, cmap='gray')
plt.title('Gray Level Slicing')
plt.axis('off')

plt.tight_layout()
plt.show()
```



**Transformation operations** help enhance the quality of the image by applying operations like log, inverse log and power on the entire image. Different type of transformation yields different results.

### 1. Power-Law (Gamma) Transformation

Gamma correction is used for adjusting brightness. The formula of power or gamma transformation is:

$$s = c \cdot r^{\gamma}$$

where c and gamma are constant.

#### Python Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def gamma_correction(image, gamma):
    # Normalize the image to range [0, 1]
    normalized_image = image / 255.0

    # Apply the gamma correction
    gamma_corrected = np.power(normalized_image, gamma)
```

```
# Scale back to [0, 255]
gamma_corrected = np.uint8(gamma_corrected * 255)

return gamma_corrected

# Load the image in grayscale
image = cv2.imread('download.jpeg', 0)

# Define different gamma values for experimentation
gamma_values = [0.5, 1.0, 1.5, 2.0]

# Plot original and gamma corrected images
plt.figure(figsize=(12, 8))

# Display original image
plt.subplot(2, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

# Apply Gamma Transformations for each gamma value
for i, gamma in enumerate(gamma_values):
    gamma_image = gamma_correction(image, gamma)

    # Display the gamma corrected image
    plt.subplot(2, 3, i + 2)
    plt.imshow(gamma_image, cmap='gray')
    plt.title(f'Gamma = {gamma}')
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Original Image



Gamma = 0.5



Gamma = 1.0





## 2. Log Transformation

Log transformation enhances darker pixels in an image. The formula of log transformation is:

$$s = c * \log(1 + r)$$

where  $c$  is a scaling constant and  $r$  is the pixel intensity.

$$c = 255 / \log(1 + r_{\max})$$

### Python Code:

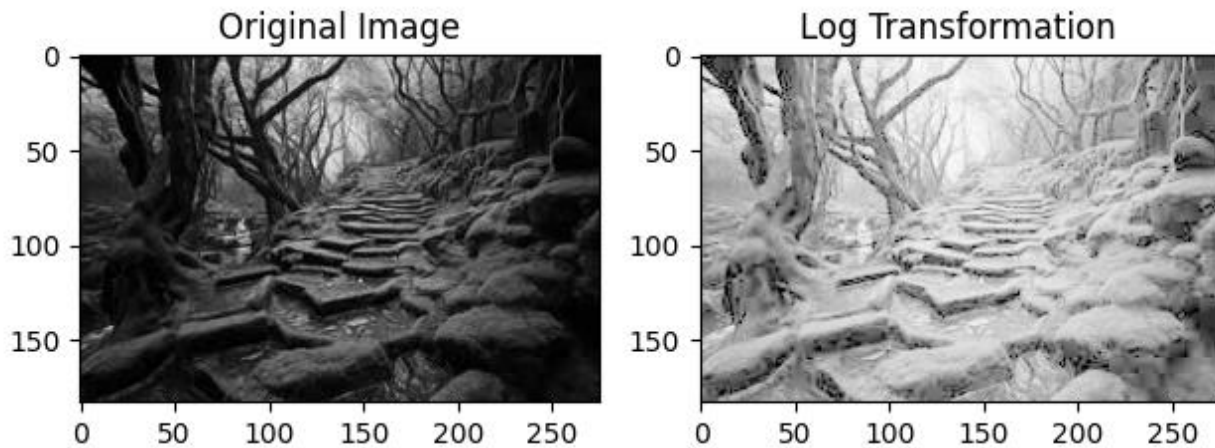
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image in grayscale
image = cv2.imread('download.jpeg', 0)
c = 255 / np.log(1 + np.max(image))
log_transformed = c * (np.log(1 + image))

# Normalize to [0, 255]
log_transformed = np.array(log_transformed, dtype=np.uint8)

# Display the result
plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')

plt.subplot(1, 2, 2)
plt.imshow(log_transformed, cmap='gray')
plt.title('Log Transformation')
plt.tight_layout()
plt.show()
```



### 3. Image Negative

This technique inverts the intensity of pixels. The formula for inverse transformation is:

$$s=(L-1)-r$$

where L is the maximum pixel value, typically 256 for 8-bit images.

#### Python Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

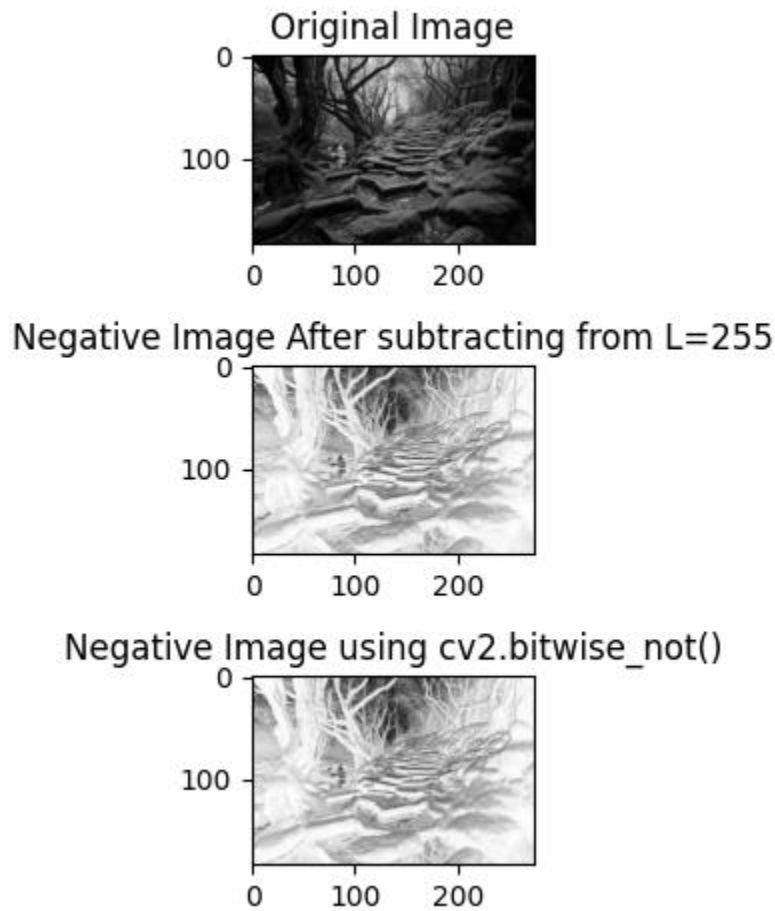
# Load the image in grayscale
image = cv2.imread('download.jpeg', 0)

# Perform negative transformation
negative_image1 = 255 - image
# Perform negative operation using cv2.bitwise_not()
negative_image2 = cv2.bitwise_not(image)
# Display original and negative image
plt.subplot(3, 1, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')

plt.subplot(3, 1, 2)
plt.title('Negative Image After subtracting from L=255')
plt.imshow(negative_image1, cmap='gray')

plt.subplot(3, 1, 3)
plt.title('Negative Image using cv2.bitwise_not()')
plt.imshow(negative_image2, cmap='gray')
plt.tight_layout()
plt.show()
```



**Some Useful Commands:**

1. To calculate the mean of 2D array using NumPy: `my_mean = numpy.mean (my_array)`
2. To calculate min (or max) of an array: `my_min = numpy.amin(my_array)`
3. To calculate the power of an array using NumPy:
4. `gamma_corrected = numpy.array (C * (image / max_gray_level) ** gamma, dtype=uint8)`
5. To obtain percentile value. `percentile_array = numpy.percentile(my_array, percentile)`
6. To change data type of array. `my_array = my_array.astype(numpy.uint16)`

**Lab Tasks:**

**1:** Apply contrast stretching on the image provided in lab by setting 5th and 95th percentiles of the input values to 0 and 255 respectively.

**2:** Apply the following transformation on an contrast enhanced image.

|           | <b>Pixel values range</b> | <b>Output pixel value</b> |
|-----------|---------------------------|---------------------------|
| <b>a)</b> | <b>Less than mean</b>     | <b>0</b>                  |
|           | <b>Greater than mean</b>  | <b>255</b>                |
| <b>b)</b> | <b>Less than mean</b>     | <b>255</b>                |
|           | <b>Greater than mean</b>  | <b>0</b>                  |
| <b>c)</b> | <b>±20 mean</b>           | <b>0</b>                  |
|           | <b>Otherwise</b>          | <b>255</b>                |

**3:** After obtaining constrast streatched images from task 1 .

Apply Power Law transformation for the following values of  $\gamma$  (0.2, 0.5, 1.2 and 1.8) . Make sure to adjust data types accordingly.